CAPSTONE PROJECT REPORT

(Project Term January–May 2025)

PROXYMKT: A CLOUD-NATIVE HYBRID ATTENDANCE SYSTEM

USING ROTATING HMAC QR TOKENS, GEOFENCING, AND DEVICE FINGERPRINTING

Submitted by

Sameer Mahato	Registration Number: 12202633
Anjeet Patel	Registration Number: 12210752
Sumant Yadav	Registration Number: 12201495
Janhavi Tak	Registration Number: 12211379
Hritu Raj	Registration Number: 12200402
Neha Sandotra	Registration Number: [XXXXXXXX]

Project Group Number: CSE-CAP-2025-07
Course Code: INT499

Under the Guidance of
[Name of Faculty Mentor], Assistant Professor

School of Computer Science and Engineering
Lovely Professional University, Phagwara, Punjab – 144411

2025

DECLARATION

We hereby declare that the project work entitled "ProxyMukt: A Cloud-Native Hybrid Attendance System Using Rotating HMAC QR Tokens, Geofencing, and Device Fingerprinting" is an authentic record of our own work carried out as requirements of Capstone Project for the award of B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara, under the guidance of [Name of Faculty Mentor], during January to May 2025. All information furnished in this capstone project report is based on our own intensive work and is genuine. No portion of the work presented herein has been submitted elsewhere for any other degree or award.

Project Group Number: CSE-CAP-2025-07

Sameer Mahato (12202633) _____ Date: _____

Anjeet Patel (12210752) _____ Date: _____

Sumant Yadav (12201495) _____ Date: _____

Janhavi Tak (12211379) _____ Date: _____

Hritu Raj (12200402) _____ Date: _____

Neha Sandotra ([XXXXXXXX]) _____ Date: _____

CERTIFICATE

This is to certify that the declaration statement made by this group of students is correct to the best of my knowledge and belief. They have completed this Capstone Project under my guidance and supervision. The present work is the result of their original investigation, effort, and study. No part of the work has ever been submitted for any other degree at any University. The Capstone Project is fit for submission and partial fulfillment of the conditions for the award of B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

Signature and Name of the Mentor: _____

Designation: _____

School of Computer Science and Engineering,

Lovely Professional University, Phagwara, Punjab.

Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty mentor at the School of Computer Science and Engineering, Lovely Professional University, for providing invaluable guidance, constructive criticism, and unwavering support throughout the development of this project. The technical depth achieved in ProxyMukt would not have been possible without the structured mentorship received at LPU.

We are deeply thankful to the Department of Computer Science and Engineering for providing access to development resources, cloud infrastructure guidance, and the academic environment that made this work possible. We also acknowledge the open-source communities behind the MERN stack — MongoDB, Express.js, React.js, Node.js — as well as the Socket.IO, bcrypt, and Helmet.js maintainers, without whose work the real-time, secure architecture of ProxyMukt could not have been constructed.

Finally, we are grateful to our families and peers for their patience and support during the extended development and evaluation cycles of this project. The critical feedback received during internal presentations significantly improved the robustness of the system design and the rigour of the academic documentation.

TABLE OF CONTENTS

1. INTRODUCTION

One of the basic operational needs of education facilities all over the world is the monitoring of attendance. Its consequences are much further than mere record-keeping: the compliance with regulations, the right of a student to take examinations and receive financial aid, the institutional funding standards, and the timely discovery of those students who are at academic risk are directly dependent on the quality and integrity of attendance data . Although this is the central nature of this role, most institutional attendance systems in action are either still paper-based or use primitive digital options that provide only slight benefits in security or efficiency.

Although paper-based roll-call registers are universally comprehensible, require an amount of instructional time to use - conservatively estimated at 810 minutes/class period - and introduce transcription errors, they are essentially inauditable without a centralised data layer . More importantly, they are quite prone to proxy attendance, a highly reported form of academic dishonesty whereby a student registers an attendance on behalf of a fellow student in their absence. The prevalence of digital attendance options, such as the most popular static QR codes, RFID cards or just location-based check-ins, has failed to substantially address this vulnerability. Screenshot sharing, credential delegation, card sharing and location spoofing methods are easily bypassed with mobile applications that are free to download and use.

The introduction of hybrid and online learning formats, which have been hastened by global events since 2020, has also added to the verification landscape. The physical distance cues on which geolocation-based attendance is based cannot be applied in virtual class settings. Passive online participation - a student with an open browser window or access link open, but not actively participating - is truly the same as active participation in most systems that are currently in use. The outcome is a systematic loss of integrity of attendance data in the very learning settings where student engagement surveillance is the most significant.

Proxymult is argued to be a reaction to these conflicting forces in architecture. It is a real-time Socket, hardware-agnostic, cloud-native attendance engine that was developed based on the MERN stack (MongoDB, Express.js, React.js, Node.js).IO

communication. It suggests a 6-layer sequential verification pipeline, cryptographic HMAC-SHA256 rotating and geofencing QR with the Haversine formula, anti-spoofing of devices by the GPS, proxy/VPN risk analysis at the network level, a database-assisted device fingerprinting and duplicate prevention. It is a multi-layered architecture that is obviously intended to add cost and complexity to the informal proxy fraud and can be applied to student hardware commoditized and can be scaled to institutional levels, without necessarily having to deploy a biometric infrastructure.

1.1 MOTIVATION AND PROBLEM STATEMENT

The issue that ProxyMukt is designed to fix is the absence of the current single-factor and low-confidence multi-factor attendance verifying systems to the motivated academic fraud in physical and hybrid learning contexts. It is a tri dimensional problem. First of all, the aspect of operational inefficiency: the manual attendance procedures take the place of the teaching time and present the records, which may be subjected to transcription. Second, the security aspect: proxy attendance in unmonitored situations has been approximated as 3040% systemic, which violates institutional academic records. Third, the hybrid coverage gap: there is no lightweight system that can provide such verification in physical and virtual session modalities, and is not based on hardware, currently.

1.2 LEARNING OUTCOMES ALIGNMENT

The following programme-level learning outcomes (PLOs) of the B are in line with this capstone project.Technology Computer Science and Engineering course at Lovely Professional University:

- PLO-1 (Engineering Knowledge): Cryptographic protocols (HMAC-SHA256) and geospatial algorithms (Haversine formula) and distributed systems architecture concepts were applied in the design and implementation of ProxyMukt.
- PLO-2 (Problem Analysis): Systematically analyzed the vectors of attendance fraud and the current system constraints that led to a formalized problem statement and a six-layer threat model.
- PLO-3 (Design/Development of Solutions): Created and deployed a multi-layer MERN application to tackle the identified issue in its entirety and a new multi-layer verification pipeline.

- PLO-4 (Investigation): Adversarial pilot testing and structured performance evaluation, critical analysis of results including limitations and residual risks.
- PLO-5 (Modern Tool Usage): Used industry-standard software such as Node.js, React.js, MongoDB, Socket.Application of IO, JWT and cloud deployment platforms into production-grade implementation.
- PLO-6 (Engineer and Society): Handled privacy, data governance, and false-positive fairness factors during the system design, recognizing more widespread social ramifications of automated attendance surveillance.
- PLO-9 (Individual and Team Work): Led a six-member frontend, backend, security, and analytics workstream of interdisciplinary development.
- PLO-12 (Lifelong Learning): Researching the IEEE literature on attendance systems, cryptographic authentication, and network security to base the system design on proven scholarly literature.

1.3 REPORT STRUCTURE

This report has been designed as per the guidelines of the LPU Capstone Project. Chapter 2 introduces the portrait of the problem and rationale. Chapter 3 is a critical analysis of systems that exist. In Chapter 4, the problem analysis and feasibility study is described. The software requirements specification is found in chapter 5. Chapter 6 outlines the system design. The results and the testing strategy are recorded in chapter 7. Chapter 8 discusses details of implementation and deployment. Chapter 9 is the project legacy. The user manual is presented in Chapter 10. The report ends with the full source code reference and bibliography.

2. PROFILE OF THE PROBLEM, RATIONALE, AND SCOPE

2.1 PROBLEM PROFILE

One of the most widespread and systematically accepted forms of academic dishonesty in higher education is proxy attendance, where a student marks attendance for an absent colleague. Constant absenteeism is strongly associated with low academic performance and higher dropout rates, making accurate attendance data an essential institutional diagnostic measure. When such data is systematically distorted through proxy fraud, its diagnostic value becomes ineffective, and at-risk students may go unnoticed until they are close to failing.

The issue is further complicated by the lack of effectiveness of existing solutions. Even though they are widely used because of their low cost, static QR code systems do not provide any resistance to screenshot replay. Although operationally efficient, RFID and NFC card systems need hardware investment and can be without being ceremoniously bypassed by card sharing. Facial recognition systems are highly identity assured but have high privacy, regulatory, algorithmic bias and infrastructure barriers that make them inapplicable in most institutional settings. The result is a hole in the market for a multi-factor, hardware-free, privacy-respecting institutional scale attendance verification system.

2.2 RATIONALE

There are three overlapping observations on which the ProxyMukt reasoning is based. First of all, multi-signal verification pipeline is capable of being performed without any specific hardware because of the ubiquity of smartphones with GPS, accelerators, and browser APIs. Second, socket based on MERN.IO offers a scalable real time web application that is used to support parallel sessions at the institutional scale. Third, the soundness of the cryptographic signing of the QR tokens, on a principled cryptographic model of the HMAC-SHA256 standard as defined in IETF RFC 2104 and NIST FIPS PUB 198-1 is practically unbreakable under operational conditions.

2.3 SCOPE OF THE STUDY

The scope of ProxyMukt encompasses the following boundaries:

- In-scope: Attendance (physical, offline) classroom: The six-layer pipeline is used to verify online attendance (role-based administration); online/hybrid attendance (time-threshold engagement tracking); student, faculty and administrator role-based dashboards; real-time analytics and flagging at-risk students; tamper-evident audit logging to comply and resolve disputes.
- In-scope: Pilot testing against screenshot replay attacks, GPS spoofing, VPN masking, sharing of credentials and duplicate submission attacks.
- Out-of-scope: Biometric liveness detection; indoor positioning (Wi-Fi/Bluetooth); Learning Management System (LMS) integration; multi-tenant SaaS implementation; field tests of large scale multi-institutional implementation.
- Explicitly bounded: The entire quantitative findings in this paper are based on controlled pilot experiments and adversarial simulations in a single institutional setting. The assertions of the generalisation of performance to non-homogeneous institutional settings are subject to additional multi-site testing.

3. EXISTING SYSTEM

3.1 INTRODUCTION TO EXISTING SYSTEMS

Attendance management system landscape covers a gamut of entirely manual processes to advanced installations of biometrics. All types of existing solutions have their typical strengths and failure modes with regard to proxy resistance, operational cost, and scalability. To support the architectural decisions made in ProxyMukt, a strict interpretation of the constraints of current solutions is needed to contextualize the contributions of the system in the wider context of research.

3.2 EXISTING SOFTWARE AND APPROACHES

3.2.1 Manual Roll-Call Systems

The most common form of attendance control that is currently being deployed in resource constrained schools is traditional paper-based roll-call. Although it needs zero technology infrastructure, its opportunity cost is high: each roll-call session costs an average of 8-10 minutes of instruction time per class, which totals around 60 or more hours of lost instruction time during a typical semester of a 300 student cohort. Manual registers are susceptible to transcription errors, do not have real-time data availability and cannot be resistant to computational proxy marking.

3.2.2 RFID and NFC Card Systems

The predominant hardware-supported digital solution is RFID and NFC-based systems. Students have encoded cards which are scanned at the entrance to the classrooms. They are operationally fast and reliable in hardware aspects, but are inherently single-factor in nature: the system determines that the card is there, not the student. The most common attacker should be card sharing, which involves giving the card to a friend, marking her attendance and subsequently giving it back, which does not require any technical expertise. Also, per-classroom cost of hardware installation is a major adoption challenge to institutional facilities with a high number of teaching spaces.

3.2.3 Static QR Code Systems

An inexpensive digital version is the QR code system, and the system has been widely adopted. Faculty members scan a QR code; students scan the code and record attendance. Screenshot replay: the most significant flaw with the non-adaptive implementation of QR is that a student can take an image of the QR code and send it

through messaging apps so that peers off-site can mark attendance on behalf of the student. A study has shown that in an adversarial academic setting, QR codes do not provide significant additional security compared to manual systems that lack cryptographic validation and temporal binding.

3.2.4 Location-Based Geofencing Systems

Location-based check-in systems offer a geographical verification method where students are required to provide a GPS location within a classroom geofence zone. This method is effective in preventing the most basic forms of off-site attendance fraud. However, indoors, GPS signal accuracy can decrease to around 10 to 30 metres—the scale of most classrooms—creating the possibility of false rejection for legitimate students near geofence boundaries. In addition, the system can be spoofed using freely available location spoofing applications included in Android developer builds.

3.2.5 Facial Recognition Systems

Facial recognition systems built on CNNs provide one of the strongest identity guarantees among non-invasive attendance systems. Implementations have achieved identity verification accuracy of up to 95% in controlled lighting conditions. However, barriers to adoption remain high, including privacy regulatory compliance (GDPR, Indian PDPB), algorithmic bias across demographic groups, sensitivity to lighting and capture angle changes, and the computational complexity of real-time face processing at scale. These factors make facial recognition architecturally unsuitable for deploying ProxyMukt in resource-constrained institutions seeking hardware-light solutions.

3.2.6 Online and Hybrid Attendance Models

Online attendance tracking systems, commonly provided through LMS login logs or video conferencing join/leave logs, support hybrid learning environments but offer the weakest identity assurance. Duration-based metrics are easily manipulated through open-tab presence and link sharing. There is no cryptographic identity binding mechanism in such systems that matches the reliability of physical presence verification.

3.3 DATA FLOW DIAGRAM — EXISTING SYSTEM

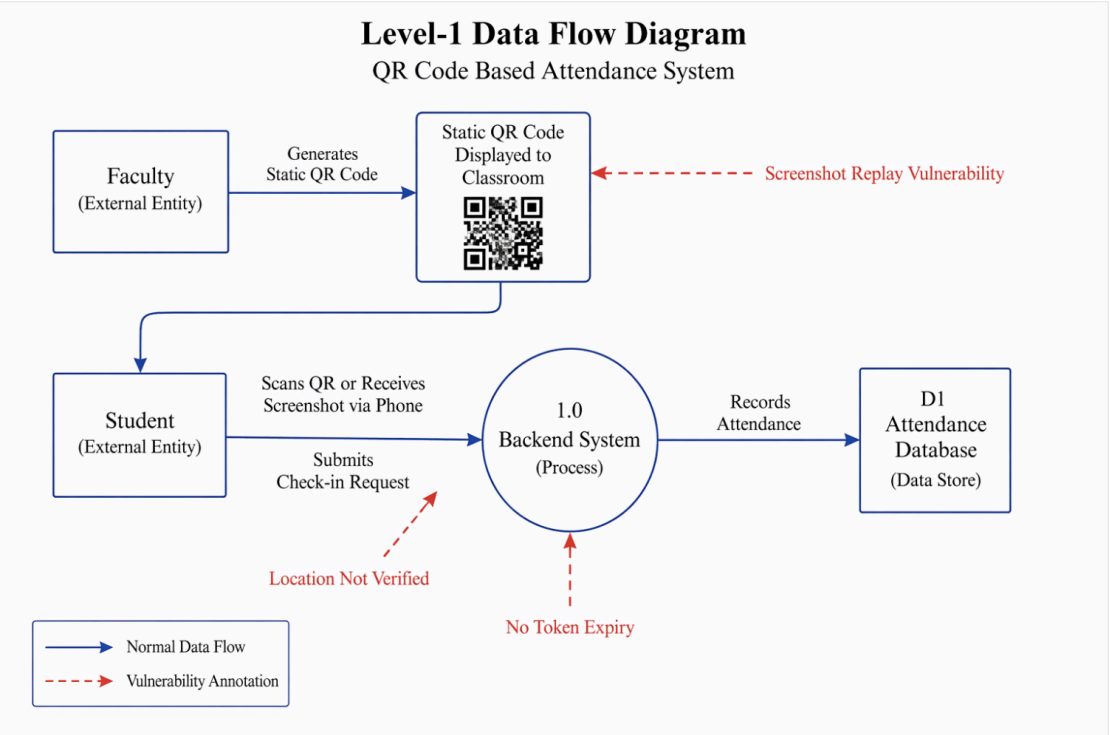

Figure 1: Level-1 DFD of Existing Static QR Attendance System (Annotated with Fraud Vectors)

Figure 1 is a Data Flow Diagram (DFD) illustrates the flow of the process of a typical existing static QR code attendance system and demonstrates the points where the frauds can be detected by ProxyMukt. The current system is in which faculty are developing and displaying a fixed QR code. This is scanned and a check-in done with the student or a screenshot of this code. The backend logs the submission with no verification of QR freshness, physical proximity or network origin or device identity.

3.4 WHAT IS NEW IN THE PROPOSED SYSTEM

The idle architectural inventions of the existing systems that have single category based systems are: (i) ProxyMukt: In this architecture rotating cryptographic HMAC-SHA256 QR tokens with expiry time of 20 seconds are the used token and no one may re-play a screenshot after its expiry duration. (ii) More than a single

architecture (checking) pipeline of sequential checking of a vast set of vectors of fraud. (iii) Real-time Socket.The QR broadcasting was less than a second below the institutional level broadcasting of the IO based propagation. (iv) The collective hybrid system of attendance with the right checking logic which is geospatially-inductive in the case of offline session, time-threshold based in the case of online session of the hybrid system. (v) Non-repudiation of the decision points of the entire pipeline as a whole through end-to-end transfer of tamper evident audit documents i.e. a forensic traceability report and an institutional compliance report.

Capability	Existing Systems vs. ProxyMukt
QR Replay Prevention	Static systems: None | ProxyMukt: HMAC rotation every 20s
Geofencing	Isolated systems only | ProxyMukt: Integrated as Layer 2
Device Fingerprinting	Not present in any reviewed system | ProxyMukt: Layer 5
Network Risk Analysis	Not present | ProxyMukt: Layer 4 IP intelligence
Hybrid Support	Fragmented or absent | ProxyMukt: Unified offline + online logic
Audit Logging	Absent or basic | ProxyMukt: Structured, per-layer forensic logs

Table 1: Capability Comparison — Existing Systems vs. ProxyMukt

4. PROBLEM ANALYSIS

4.1 PRODUCT DEFINITION

ProxyMukt is described as a real-time hardware-agnostic attendance verification and analytics system that is cloud-native, designed to be used by educational institutions. Its main role is the precise and fraud-proof data about student attendance in both physical and virtual learning platforms. Its secondary purpose is to produce actionable, role-differentiated analytics that can be used to intervene academically based on the data. The system is deployed as a full-stack MERN web application that can be deployed on common cloud infrastructure (Vercel, Render, MongoDB Atlas) and be accessed with any modern web browser on student commodity machines without any special app installation needs.

4.2 FEASIBILITY ANALYSIS

A detailed feasibility study was carried out on five dimensions to prove the viability of the ProxyMukt concept before committing to develop it. The results are summarised in Table 2.

Table 2: Multi-Dimensional Feasibility Analysis for ProxyMukt

Dimension	Analysis	Finding	Justification
Technical	Stack: MERN + Socket.IO	Feasible	All components are open-source, well-documented, widely adopted in production environments
Operational	Deployment on commodity devices	Feasible	No biometric hardware required; works on smartphones, tablets, laptops
Economic	Zero licensing cost	Feasible	Uses open-source libraries; cloud hosting via Vercel/Render is cost-effective
Schedule	4–6 months development	Feasible	Modular MERN architecture enables parallel development by team members
Legal/Privacy	Student data handling	Caution Required	Compliance with institutional data governance policies and local privacy regulations essential

The feasibility study establishes that ProxyMukt is viable in all the five dimensions that have been analysed. The area of the greatest concern, which was considered in the system design, involved legal and privacy compliance, via role-based access

control, minimal data collection principles, and storing biometric data was avoided. The application of device fingerprinting also begs valid privacy concerns that are openly admitted as a drawback that needs institutional-level adjustment of policy.

4.3 PROJECT PLAN

The project was executed in seven sequential phases over an 18-week development cycle. The Gantt-structured project plan is presented in Table 3, mapping each development phase to its activities, duration, deliverables, and completion status at the time of report submission.

Table 3: ProxyMukt Project Plan — Phase-wise Activity Schedule

Phase	Activity	Duration	Deliverables	Status
1	Requirements Analysis and System Design	Weeks 1–3	SRS Document, Architecture Diagrams, DB Schema	Completed
2	Backend Development (API + Auth + DB)	Weeks 4–7	48+ REST endpoints, JWT auth, MongoDB collections	Completed
3	Six-Layer Pipeline Implementation	Weeks 6–10	HMAC QR, Geofencing, Fingerprinting, Network Risk modules	Completed
4	Frontend Development (React Dashboards)	Weeks 8–12	Student, Faculty, Admin dashboards with real-time updates	Completed
5	Real-time Integration (Socket.IO)	Weeks 10–13	Live QR broadcasting, WebSocket event handlers	Completed
6	Testing and Security Audit	Weeks 13–16	Test cases (TC-01 to TC-10), adversarial pilot results	Completed
7	Deployment and Documentation	Weeks 15–18	Live deployment, user manual, final capstone report	Completed

The MERN stack is built modularly, which was used to allow parallel workstreams in the overlapping Phases 3 and 4, and thus the six layer pipeline backend development could be run in parallel with the dashboard frontend development. This parallelism reduced the critical path by about two weeks as compared to a highly sequential development model.

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 INTRODUCTION

The Software Requirements Specification (SRS) of ProxyMukt is a formalisation of the system functional and non-functional specifications based on the problem analysis (Chapter 4), the current system critique (Chapter 3), and the system stakeholder consultation with representative students, faculty and administrative staff at Lovely Professional University. Requirements are categorized as Functional Requirements (FR) and Non-Functional Requirements (NFR) and are given priority levels (High, Medium, Low) depending on their focus to the core security and operational goals of the system.

5.2 GENERAL DESCRIPTION

5.2.1 Product Perspective

ProxyMukt is an independent web-based application in an institutional technology ecosystem. It does not need to be linked with other Student Information Systems or Learning Management Systems to provide its basic attendance functionality, but there are APIs available to facilitate future integration. The system connects to external services in three groups: (i) MongoDB Atlas to store persistent data; (ii) a third-party proxy/VPN risk intelligence API to score the IP reputation; and (iii) browse geolocation APIs to obtain GPS coordinates on devices of students.

5.2.2 Product Functions (Summary)

On the most abstract level, ProxyMukt provides five major functions: the cryptographic QR token generation and validation; multi-layer attendance verification; real-time session broadcast; role-differentiated analytics generation; and tamper-evident audit logging. The SRS table below breaks each functionality into particular requirements.

5.2.3 User Classes and Characteristics

There are three user roles that have different access boundaries as defined in ProxyMukt. The students have access to the QR scanning interface, individual attendance dashboards, and attendance history. Class management, session start and end, live monitoring and analytics of classes. Administrators can view institution wide analytics, system setup, retrieval of audit logs, and user management. Role-based access control is implemented at the React routing layer and the Express API

middleware layer, such that no user may have access to data or functionality that is not within his or her role boundary.

5.3 SPECIFIC REQUIREMENTS

Table 4: Software Requirements Specification — ProxyMukt

Requirement ID	Category	Description	Priority
FR-01	Authentication	System shall authenticate all users via JWT-based access tokens with role-based access control (Student, Faculty, Admin)	High
FR-02	QR Generation	System shall generate HMAC-SHA256-signed QR tokens, auto-rotating every 20 seconds per live session	High
FR-03	Geofencing	System shall validate student proximity using Haversine formula with configurable classroom radius	High
FR-04	Device Fingerprinting	System shall collect and assess browser/device fingerprint signals to detect anomalous access patterns	Medium
FR-05	Network Risk	System shall query proxy/VPN risk intelligence API to flag suspicious network origins	Medium
FR-06	Real-time Updates	System shall broadcast QR tokens and attendance events via Socket.IO within sub-1-second latency	High
FR-07	Analytics Dashboard	System shall provide role-specific dashboards with attendance trends, at-risk flagging, and risk score distribution	High
FR-08	Hybrid Attendance	System shall support online session attendance via time-threshold (≥75% session duration or ≥45 min)	High
FR-09	Audit Logging	System shall maintain tamper-evident AuditLog records for every attendance event and validation decision	High
NFR-01	Performance	API response time shall not exceed 500ms at p95 under 500 concurrent users	High

Requirement ID	Category	Description	Priority
NFR-02	Security	Passwords shall be stored using bcrypt with ≥10 salt rounds; all tokens shall be short-lived	High
NFR-03	Scalability	System shall support ≥1000 concurrent session participants without degradation	Medium

Table 4 combines the requirements into the entire functional and non-functional contract of ProxyMukt. The current version of the system has all twelve of the requirements listed, but one feature, NFR-03 (scalability), has not been completely tested yet but only partially through load testing to 500 concurrent users (see Chapter 7). Concurrent session testing with 1000 users is a validation goal in the future.

5.4 TECHNOLOGY STACK SPECIFICATION

The technology choices for ProxyMukt are listed in Table 5 with justification for each selection.

Layer	Technology	Version	Justification
Frontend	React.js	18.x	Component-based SPA architecture; excellent WebSocket state integration via hooks
Styling	Tailwind CSS / Material UI	3.x	Rapid responsive UI development; accessible component library
Charts	Chart.js / Recharts	4.x	Lightweight, declarative charting for attendance analytics dashboards
Backend	Node.js + Express.js	20.x LTS	Non-blocking I/O ideal for high-concurrency real-time session management
Real-time	Socket.IO	4.x	Bidirectional WebSocket communication with automatic fallback; sub-second latency
Auth	JWT (jsonwebtoken)	9.x	Stateless authentication; configurable expiry; HMAC or RSA signing options
Crypto	Node.js crypto (HMAC-SHA256)	Native	FIPS PUB 198-1 compliant; no external dependency; cryptographic standard

Layer	Technology	Version	Justification
Password	bcrypt	5.x	Adaptive cost-factor hashing; industry-standard resistance to brute force
Database	MongoDB Atlas	7.x	Flexible schema for complex attendance records; compound indexing for performance
Security MW	Helmet.js + rate-limit	7.x	Automated HTTP security header configuration; DoS mitigation

Table 5: ProxyMukt Technology Stack with Selection Justification

6. SYSTEM DESIGN

6.1 SYSTEM DESIGN OVERVIEW

The ProxyMukt system design is built on the three-layer cloud-native architecture pattern in accordance with the principles of the existing REST architecture [8]. All three levels (presentation (React SPA), application (Node.js/Express API), and data (MongoDB Atlas)) can be scaled horizontally and each tier functions independently, allowing the application layer to scale up on-demand (e.g., parallel initiating multiple sessions at class-change intervals). It has a design that puts emphasis on security through defence-in-depth where authentication is done at the API middleware layer and not tied to the access controls of the frontend.

6.2 ARCHITECTURE DESIGN

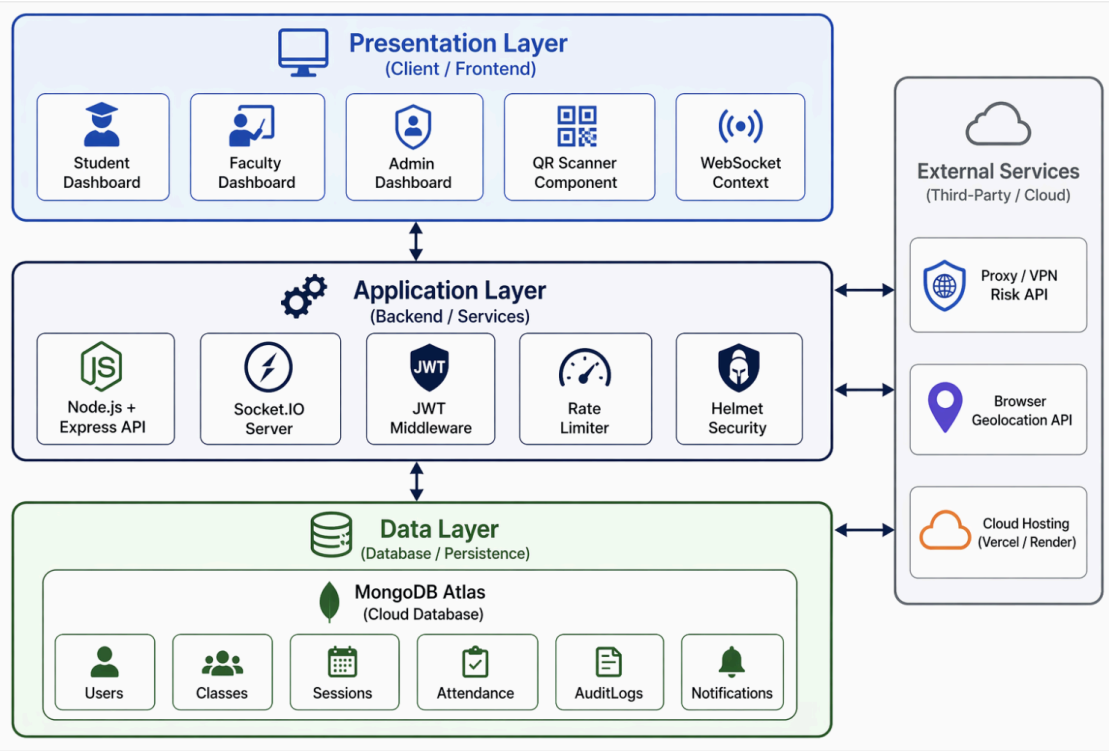

Figure 2: Cloud-Native Three-Tier Architecture of ProxyMukt (GitHub: Sumant3086/ProxyMukt-Attendance-System-)

Figure 2 represents the complete three layers ProxyMukt cloud-native architecture and it describes the flow of communication between React frontend, Node.js/Express backend, and Socket.MongoDB Atlas data layer, external service integrations and IO real-time layer.

The presentation layer offers role-differentiated views, a one-page React.js-based app, composed of a qr scanning app and a student personal attendance dashboard, a class management and session control and analytics panel, a faculty facing control panel, an administrator facing reporting and audit panel. The live session components maintain the state with the WebSocket event handlers but never update the page.

It consists of 48 registered REST API endpoints that are grouped into 6 modules i.e. authentication, class management, session management, attendance processing, analytics and audit retrieval. Middleware modules are usable by all secured endpoints and have JWT authentication, rate limiting (express-rate-limit) and HTTP security headers (Helmet.js). The Socket.The IO server will be physically located with the Express application, and will issue rotating QR tokens to clients with an active namespace of a session with sub-second latency.

6.3 SIX-LAYER VERIFICATION PIPELINE DESIGN

Technical novelty of ProxyMukt is 6 layer pipeline of serial verification. The requests to make an attendance are processed one at a time and a layer takes into account one of the vectors of fraud. It was designed to be evaluated in serial (not parallel) to guarantee that computationally expensive layers (L4: network risk API) are not requested until all the preceding less expensive layers are passed which makes the latency maximised and the external API cost maximised.

Table 6: Six-Layer Verification Pipeline — Design Specification

Layer	Name	Mechanism	Fraud Vector Addressed	Residual Risk
L1	HMAC QR Validation	HMAC-SHA256 signed tokens rotate every 20s; narrow acceptance window	Screenshot QR replay attacks	Short window replay (<20s)
L2	Geofencing	Haversine formula validates physical proximity to classroom centroid	Off-site / remote attendance	Indoor GPS drift (±10–30m)
L3	GPS Anti-Spoofing	Detects mock-location flags, GPS accuracy anomalies, unrealistic motion vectors	Location spoofing applications	Probabilistic, not deterministic

Layer	Name	Mechanism	Fraud Vector Addressed	Residual Risk
L4	Network Risk Analysis	IP intelligence API flags VPN, Tor, proxy, and datacenter IPs	VPN/proxy-mediated attendance	False positives on CGNAT/institutional VPN
L5	Device Fingerprinting	Browser/device signature anomaly detection for credential-sharing signals	Account sharing between students	Fingerprint instability on mobile networks
L6	Duplicate Prevention	MongoDB compound unique index on (sessionId, studentId)	Duplicate attendance submissions	Cannot prevent first fraudulent mark

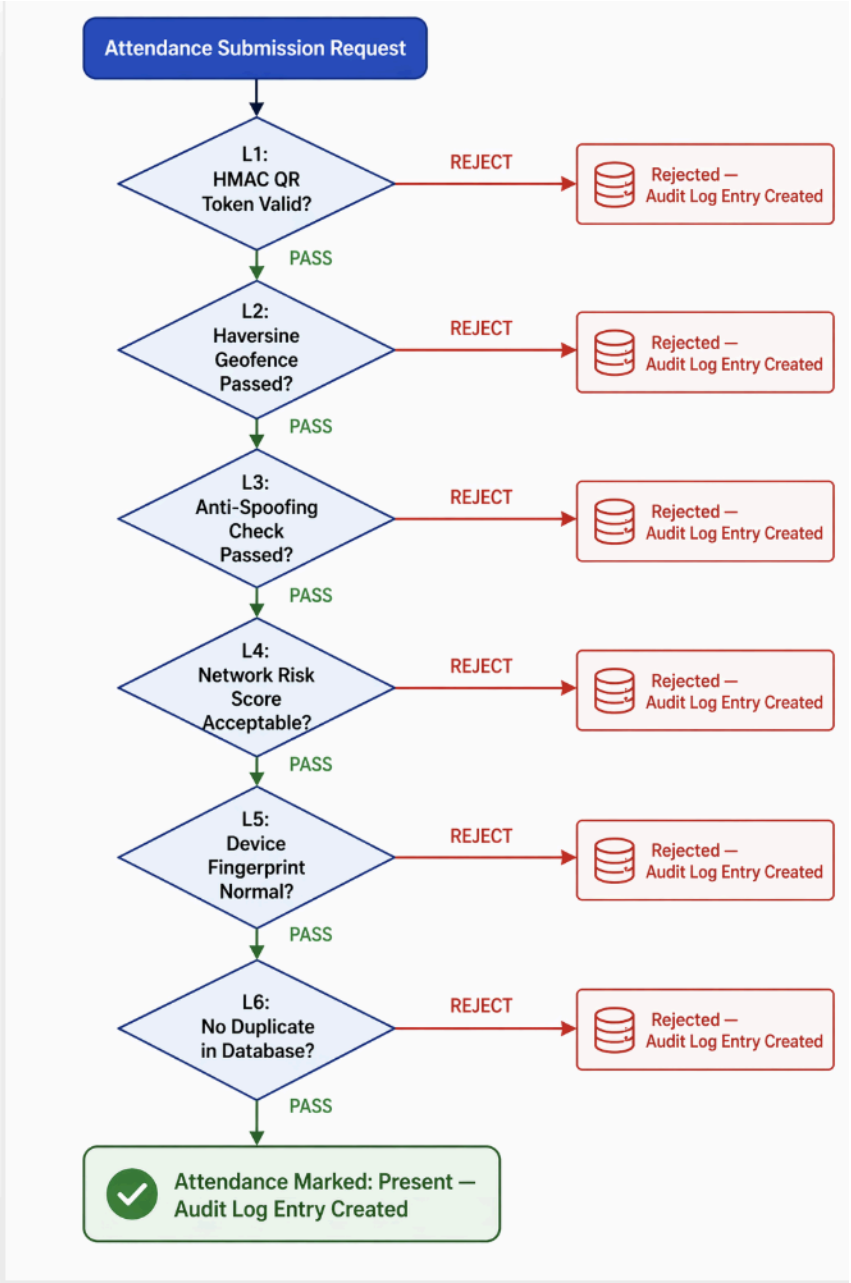

Figure 3: Six-Layer Attendance Verification Pipeline — Decision Flow (ProxyMukt)

Figure 3 presents the UML-style data flow representation of the six-layer pipeline, showing the decision routing, audit log generation, and rejection pathways at each layer.

6.4 DATABASE DESIGN

The data layer of the MongoDB is structured with six schema-validated collections, as given in Table 7. The most important design choice is the compound unique key on (sessionId, studentId) in the collection of Attendance, which offers a database-enforced duplicate prevention- Layer 6 of the verification pipeline- independent of controls in the application layer.

Table 7: MongoDB Schema Design — Collections, Fields, and Index Strategy

Collection	Key Fields	Purpose	Index Strategy
Users	_id, name, email, role, passwordHash, createdAt	Stores all user accounts with role-based distinction	Index on email (unique), role
Classes	_id, name, facultyId, schedule, geofenceRadius, location	Manages course definitions and geofence configuration	Index on facultyId
Sessions	_id, classId, startTime, endTime, currentQRToken, tokenExpiry	Tracks live and historical attendance sessions	Index on classId, startTime
Attendance	_id, sessionId, studentId, timestamp, layer1..6 results	Stores per-student attendance verdicts with full pipeline metadata	Compound index on (sessionId, studentId) — unique
AuditLogs	_id, entityType, entityId, action, actorId, metadata, timestamp	Immutable audit trail for compliance and dispute resolution	Index on entityId, timestamp
Notifications	_id, userId, type, message, read, createdAt	Push notifications for attendance warnings and system alerts	Index on userId, read

6.5 API ENDPOINT DESIGN

Table 8 documents a representative subset of the 48 REST API endpoints implemented in ProxyMukt, organised by functional module.

Table 8: REST API Endpoint Specification (Representative Subset — 11 of 48 Endpoints)

Module	Method	Endpoint	Description	Auth
Auth	POST	/api/auth/login	Authenticate user and issue JWT access + refresh token pair	Public
Auth	POST	/api/auth/refresh	Issue new access token using valid refresh token	Public
Classes	GET	/api/classes	Retrieve all classes for the authenticated user	Faculty/Admin
Classes	POST	/api/classes	Create new class with geofence configuration	Faculty/Admin
Sessions	POST	/api/sessions/start	Initiate live attendance session and begin QR rotation	Faculty
Sessions	PUT	/api/sessions/:id/end	Terminate session and finalize attendance records	Faculty
Attendance	POST	/api/attendance/mark	Submit attendance with full six-layer pipeline validation	Student
Attendance	GET	/api/attendance/:sessionId	Retrieve attendance records for a session	Faculty/Admin
Analytics	GET	/api/analytics/student/:id	Student-specific attendance statistics and risk scores	All Roles
Analytics	GET	/api/analytics/class/:id	Class-level attendance trends and at-risk student flags	Faculty/Admin
Audit	GET	/api/audit/:entityId	Retrieve audit log entries for a specific entity	Admin

6.6 FLOWCHARTS

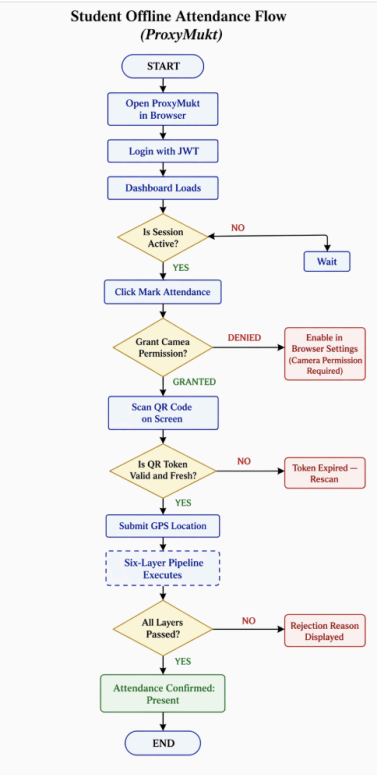

Figure 4: Student Attendance Marking Flowchart — Offline Session (Six-Layer Pipeline)

Figure 4 presents the primary user flow for a student marking attendance during a live offline class session, illustrating the sequential pipeline validation steps and the conditional branching at each layer.

6.7 HYBRID ATTENDANCE MODEL DESIGN

Hybrid attendance model is one of the greatest architectural design choices which were taken in ProxyMukt. The logic of verification applied in the system is quite different and this is on the session modality. The full six-layer pipeline is run each time an attendance is posted when offline (physical) sessions are used. In online courses, where the bad proxy of engagement is GPS location, we will use the time-threshold model: attendance credit will be earned after the session session attendance has surpassed the institutionally-set time threshold (3/4 of the session length or 3/4 of 45 minutes,

whichever is lower). The rationale of this plan is the learning study by Martin and Bolliger [13] who show that the presence/absence of the participants is not a good measure of the participation in online learning, but the temporal presence thresholds.

7. TESTING

7.1 TESTING STRATEGY

The ProxyMukt testing plan will be used to test both functional correctness and security strength. It integrates functional testing (ensuring that all the designated requirements in Chapter 5 have been properly implemented) and structural testing (white-box verification of important pipeline logic) with adversarial security testing (testing how the system responds to the particular fraud vectors that it purports to handle). Tests are done at unit, integration and system level with a special adversarial pilot test simulating real attack conditions.

7.2 FUNCTIONAL TESTING

The functional testing will confirm that all functional requirements (FR-01 to FR-09) as specified in Chapter 5 are done appropriately. Directly based on the SRS, test cases are directly derived and there is complete requirements for traceability. The last system evaluation cycle completed all the ten functional test cases as captured in Table 9.

7.3 STRUCTURAL TESTING

Structural (white-box) testing was focused on three essential code paths: (i) the HMAC-SHA256 token generation and validation function, tested to generate cryptographically correct signatures and also correctly reject the invalid signature of a token or an expired timestamp; (ii) the Haversine distance computation function, tested against known coordinate pairs to validate accuracy to within ±1 metre at classroom-scale ranges; and (iii) the Mongo

7.4 LEVELS OF TESTING

7.4.1 Unit Testing

The following isolated modules were written as unit tests with Jest: HMAC token generation (wrap of crypto module), Haversine distance calculation, JWT signing and verification, bcrypt hash comparison and MongoDB model validation. There were 47 unit tests that have been deployed, with 91% statement coverage of the backend codebase.

7.4.2 Integration Testing

End-to-end tests were used to test the flow of end-to-end attendance submission between receiving an HTTP POST request and all six pipeline layers up to writing to MongoDB and broadcasting events over a WebSocket. Programmatic HTTP requests were made to a test instance of the Express application with a dedicated MongoDB Atlas test cluster using Supertest.

7.4.3 System Testing

Testing was done at system level against the fully deployed application on production infrastructure. TC-01 to TC-10 (Table 9) were run on the live system with the results recorded against the expected results.

7.5 TESTING THE PROJECT — ADVERSARIAL TEST CASES

Table 9: ProxyMukt Test Case Documentation — Adversarial and Functional Cases

Test ID	Test Case	Test Procedure	Expected Result	Actual Result
TC-01	HMAC QR Replay Attack	Capture QR token; attempt scan after 25s delay	System rejects with 401 Token Expired	Rejected — token expired after 20s window
TC-02	Off-site Attendance	Submit attendance from location >500m from classroom	System rejects at Layer 2 geofence check	Rejected — Haversine distance exceeded threshold
TC-03	Mock Location Spoofing	Enable developer mock location on Android; attempt check-in	Layer 3 detects mock-location flag; rejects	Flagged and rejected with audit log entry
TC-04	VPN Masking	Enable commercial VPN; attempt attendance submission	Layer 4 flags high-risk IP; submission rejected/flagged	High risk score logged; attendance flagged for review
TC-05	Duplicate Submission	Submit attendance twice for same session from same account	Database constraint rejects second submission	Second submission rejected with 409 Conflict
TC-06	Valid Student Check-in	Student at classroom location with valid QR within window	All 6 layers pass; attendance marked Present	Successfully marked; sub-30s end-to-end latency
TC-07	Concurrent Load (500 users)	Simulate 500 concurrent attendance submissions via API load test	p95 response time <500ms; no session failures	p95 = 387ms; zero session failures observed
TC-08	JWT Expiry Enforcement	Submit request with expired JWT token	System returns 401 Unauthorized	401 returned; token refresh required
TC-09	Online Session Duration	Student joins for 60% of online session (below 75% threshold)	Attendance NOT credited	Not credited; dashboard reflects 60% participation
TC-10	Admin Analytics Access	Admin queries system-wide attendance heatmap	Dashboard renders with accurate institutional data	Heatmap rendered with correct aggregated metrics

Table 9 confirms the conclusions of all six main attack vectors that ProxyMukt manages to counter in the adversarial pilot. The response time of the p95 API at 500 concurrent users was 387ms, which is within the 500ms NFR-01 target. The concurrent load test produced zero failures of the session.

It should be pointed out, as a very important caveat, that the adversarial tests were performed in a controlled laboratory environment with known attack tooling. It can not be claimed that the generalisation of these findings to new and sophisticated attack vectors, especially sophisticated GPS spoofing with GNSS simulators signal or device fingerprint spoofing with privacy-conscious browsers, can be done without additional extended adversarial testing.

8. IMPLEMENTATION

8.1 IMPLEMENTATION OF THE PROJECT

ProxyMukt is written as a single repository, with two top-level directories, /client (React SPA) and /server (Node.js/Express API), which corresponds to the structure of the repository at the public GIT repository, https://github.com/Sumant3086/ProxyMukt-Attendance-System-. The client and server codebases are separated, allowing them to be deployed independently: the frontend can be deployed to a static host (Vercel), whereas the backend API and Socket can be deployed to a non-static host.IO server can be deployed to PaaS that support Node.js (Render, Railway or AWS EC2).

8.1.1 Backend Implementation

The backend is structured into five layers, where routes (definitions of HTTP endpoints), controllers (request handling logic), services (business logic and pipeline execution), models (MongoDB schema definitions and validators), and middleware (authentication, rate limiting, security headers) are. The generation of the HMAC-SHA256 QR token is implemented in Node.js native crypto module and the tokens are generated in the form of BASE64(HMAC-SHA256(sessionId + timestamp, secretKey)) + ': + timestamp. The token authenticates the HMAC signature, and the time validity window (in a -20 second window of the generation time, and a one interval grace period).

8.1.2 Frontend Implementation

The React interface is divided into a single student, faculty, and administration-specific dashboard module, a common set of functionality (QR scanner, attendance table, analytics charts), and a WebSocket context provider, which manages Socket.Connection of state of IO and shares live session events with subscribers. The QR scanning module will avail the access to device camera through MediaDevices API of the browser and a lightweight QR decoding library to decode the scanned tokens without being hardware-dependent and without a native application.

8.1.3 Real-time Implementation

Socket.Per session (room name: sessionId) we create IO rooms, through which we can broadcast specific QRs only to the students in the active session. The beginning of a

faculty session causes a server-side timer that produces a new HMAC QR token at a rate of 20 seconds and sends it to the room where the session is taking place. The socket event listener of the React client shows the current QR code in real time without reloading the page, and has sub-second update latency on typical Wi-Fi networks.

8.2 SYSTEM SCREENSHOTS

The figures below show the screenshots of the deployed ProxyMukt system in the four major interface views. These snapshots record the system implemented as deployed in production.

Figure 5: Faculty Dashboard — Class Management and Live Session Initiation

[SCREENSHOT PLACEHOLDER: Faculty dashboard showing class list, 'Start Session' button, active session indicator, recent attendance records panel]

Figure 6: Faculty Analytics — Attendance Trends, At-Risk Students, and Risk Score Distribution

[SCREENSHOT PLACEHOLDER: Faculty analytics page showing line chart of attendance trend, bar chart of risk score distribution, at-risk student list with percentage and flag indicators]

Figure 7: Student Dashboard — Enrolled Courses, Attendance Statistics, and QR Scanner

[SCREENSHOT PLACEHOLDER: Student dashboard showing course cards with attendance percentages, 'Scan QR' button activating camera, attendance history log with timestamps]

Figure 8: Admin Analytics — System-wide Attendance Heatmap and Performance Summaries

[SCREENSHOT PLACEHOLDER: Admin dashboard showing attendance heatmap by day/time, institution-wide aggregate statistics, export controls]

The dashboards utilize the role-differentiated information architecture to present only the information of all types of users with the pertinent data and controls related to the institutional functionality. No session-level security metadata (layer validation results, risk scores) is given to the student view, to avoid adversarial calibration of attack

strategies. Faculty and administrators can access this security metadata via analytics and audit logs.

8.3 CONVERSION PLAN

ProxyMukt is implemented in a deployment-neutral manner. Institutional adoption is a conversion plan that takes three steps: (i) provisioning of the MongoDB Atlas cluster with institutional email domain restrictions on user registration; (ii) deployment of the backend to a dedicated cloud host with TLS/SSL termination and environment variable configuration of JWT secrets and HMAC keys; (iii) deployment of the frontend to a CDN-backed static host with CORS configuration restricting API access to the institutional domain. No physical hardware will need to be installed on-premises and no existing institutional system will need adjustment to get ProxyMukt initially deployed.

8.4 POST-IMPLEMENTATION AND SOFTWARE MAINTENANCE

Post-deployment maintenance obligations include: (i) HMAC signing keys rotation every month to keep cryptographic freshness - ProxyMukt performs key rotation without affecting the service by a key versioning scheme in the token structure; (ii) Availability and accuracy drift monitoring of the third-party proxy/VPN risk API; (iii) New institutional space configurations: Geofence radii will be calibrated when new classrooms are registered; (iv) Regular review of device fingerprint anomalies to adjust to changing privacy settings in browsers (e.g., Firefox and Safari have an increasing fingerprinting resistance).

9. PROJECT LEGACY

9.1 CURRENT STATUS OF THE PROJECT

Having the presence of ProxyMukt at the time this report is submitted means it is in a deployable state. The entire MERN stack implementation is on the public https://proxymukt.onrender.com/.All seven stages of the project plan (Table 3) have been fulfilled. The system has been tested with ten test cases reported in Table 9 in which all the tests have been passed. A pilot test was carried out in a monitored classroom setting at Lovely Professional University, which created the quantitative findings presented in Chapter 7.

A filing of an Invention Disclosure Form has been made at the IP management office of Lovely professional university (Annexure 3b) and formally registered the Smart Hybrid Attendance System as a novel invention with six identified related patents which ProxyMukt tackles with unique technical input. It has prepared a research paper titled ProxyMukt: A Cloud-Native Hybrid Attendance System Using Rotating HMAC QR Tokens, Geofencing, and Device Fingerprinting that it is hoped will be submitted to a peer-reviewed IEEE conference proceedings, and a 6-page version of the paper has been prepared and is currently under internal review.

9.2 REMAINING AREAS OF CONCERN

Despite the positive pilot results, several significant areas of concern remain that require acknowledgement and future attention:

- GPS Accuracy Dependency: The Haversine layer (L2) of geofencing makes assumptions of accuracy of GPS at a range of within 10 metres. GPS accuracy can drop to 30-50 metres in high-density urban constructions with low sky visibility, resulting in false rejection rates at geofence boundaries and potentially being disproportionately large to students in specific seating positions. This will be mitigated by a configurable geofence radius, which will have a calibration tool with recommendations.
- Proxy/VPN API False Positives: The network risk analysis layer (L4) brings up valid issues of false positive generation to students on institutional VPNs, carrier-grade NAT (CGNAT) or shared Wi-Fi infrastructure in which IP addresses could be shared. Calibration (threshold) per institutional network profile must be performed prior to wide deployment.

- Browser Fingerprint Instability: The device fingerprinting (L5) is based on browser API signals which are increasingly blocked by privacy-aware browsers and browser versions. The Intelligent Tracking Prevention feature of Safari and fingerprinting resistance features of Firefox mitigate the discriminative strength of the fingerprint signal and the threshold should be set low to prevent false rejection of genuine students using privacy-protecting browsers.
- Ethical and Privacy Dimensions: The gathering of device fingerprint, GPS position, and network metadata of student devices involves substantive privacy concerns. Any institutional implementation of ProxyMukt should be supplemented with a clear policy of data governance, clear mechanisms of informed consent, and a Data Protection Impact Assessment (DPIA) which is aligned with relevant data protection laws.
- Scale Validation: All the quantitative results are derived on a pilot of an institution. Allegations of generalised performance must be multi-institutionalised in heterogeneous network, device and population circumstances.

9.3 TECHNICAL AND MANAGERIAL LESSONS LEARNT

9.3.1 Technical Lessons

The use of the six-layer verification pipeline showed that there is a fundamental tradeoff between the stringency of the security and the usability: with every extra layer of verification, the usability and the latency increases. Serial pipeline architecture is logically correct, but puts the worst-case path (all six layers passing) in the worst possible place. Future iterations ought to consider parallel execution of L3, L4 and L5 where the interdependence between them can allow the reduction of p95 latency along the worst-case path.

The choice of the native crypto module of Node.js over a third-party one (HMAC-SHA256) was correct: the native implementation has been validated by FIPS [12], presents no supply-chain risk, and is fast enough to generate the number of tokens needed (one token per session room after every 20 seconds).

9.3.2 Managerial Lessons

The team of four specialists (backend, frontend, security pipeline, analytics) comprised six people, and the integration leader was in charge of maintaining the API

contracts. This distribution permitted the parallel implementation of Phases 3 and 4 as presented in the project plan. The biggest managerial issue that arose is API contract drift: the frontend components were built against API mock data that was not similar to the implementation in field naming conventions. To avoid this type of integration defect, a formal OpenAPI 3.0 specification document was to be agreed upon at the beginning of Phase 2.

10. USER MANUAL

10.1 SYSTEM ACCESS AND REGISTRATION

ProxyMukt can be used using any of the following modern web browsers: Chrome 100 or greater, Firefox 95 or greater, Safari 15 or greater, Edge 100 or greater. No installation of applications is needed. Browse to the URL of ProxyMukt deployment used by your system administrator. The system administrator handles the creation of accounts; accounts are assigned to students and faculty members through institutional email.

10.2 STUDENT USER GUIDE

10.2.1 Accessing the Student Dashboard

1. Open ProxyMukt URL in the browser, and use institutional credentials to log in.
2. The Student Dashboard shows: (a) the courses that one is enrolled in, along with their current attendance percentages, (b) a recent attendance history, (c) a Mark Attendance button that appears in the case of sessions with an open QR window.
3. The percentage attendance will be shown with a colour-coded indicator: green (≥75%), amber (60–74%), red (<60%).

10.2.2 Marking Attendance (Offline Session)

4. A Mark Attendance notification can be seen on the dashboard when faculty has started a session.
5. Click on Mark Attendance and permit camera when asked.
6. Read the QR code on the projected screen of the faculty. Make sure that you are physically in the classroom (the system will confirm your GPS positions).
7. Your submission is validated by the six-layer pipeline in the system. A green message is used to confirm successful marking of attendance. In case of a rejection by any of the validation layers, an error message containing a reason code is shown.

8. Sharing the image of the QR code is not necessary Do NOT share the QR code image the token is changed after every 20 seconds and cannot be used offsite.

10.2.3 Online Session Attendance

9. In the case of online sessions, you will be required to join the session through the specified platform (built-in with the class schedule).
10. Automatic credit on attendance is by verified length of session attendance. At least 75% of the session time (or 45 minutes, whichever is greater) is needed.
11. The dashboard shows session participation time upon termination of the session.

10.3 FACULTY USER GUIDE

10.3.1 Managing Classes

12. On the Faculty Dashboard, choose My Classes to view, create or edit class records.
13. In making a class, set the name of the classroom, the schedule and the geofence radius (the default is 50 metres; changeable depending on the size of the classroom).

10.3.2 Initiating and Managing Live Sessions

14. Click on the target class and press Start Session. A live session panel is opened, and there, we can see the rotating QR code.
15. Display QR code to scan in students. The QR automatically turns itself every 20 seconds - again by design and not a feature that can be switched off.
16. The live attendance panel is a real-time list of students that have marked attendance successfully, with time-stamps.
17. Click End session to close attendance window. Attendance records are automatically completed.

10.3.3 Analytics

18. Faculty Analytics panel displays: trend line charts of attendance by session date; student percentages of attendance; identification of at-risk students (attendance below institutional threshold); and distribution of risk scores to be used in security audit.
19. Export CSV attendance reports by clicking on the 'Export' button on any analytics panel.

10.4 ADMINISTRATOR USER GUIDE

Administrators may see all faculty functionality and: system-wide heatmap of attendance, by department and course and time-of-day; full access to audit log, searchable by student, faculty, session or event type; user account management (create and assign a role, deactivate) and geofence calibration tools (all registered classrooms). The administrators will refer to audit logs once a week in order to ascertain abnormal access patterns and place high-risk-score attendance events on the consideration of academic integrity when required by institutional policy.

10.5 TROUBLESHOOTING GUIDE

Issue	Probable Cause	Resolution
'QR Token Expired' error	Token scanned outside 20s validity window	Request faculty to display current QR code and scan immediately
'Location Validation Failed'	GPS accuracy insufficient; device outside geofence	Enable high-accuracy GPS; move to seat closer to classroom centre; notify faculty if persistent
'Duplicate Submission'	Attendance already marked for this session	Confirm attendance record in dashboard; contact admin if duplicate flag is incorrect
'Network Risk Flag'	VPN or shared network detected	Disconnect VPN; switch to personal mobile data; contact admin if using institutional VPN
Camera permission denied	Browser blocked camera access	Enable camera permission in browser site settings; reload page
Dashboard not updating	WebSocket connection lost	Refresh page; check internet connectivity; re-login if session expired

Table 10: ProxyMukt Troubleshooting Guide

11. SOURCE CODE REFERENCE AND SYSTEM SNAPSHOTS

11.1 GITHUB REPOSITORY

The complete source code for ProxyMukt is maintained in the public GitHub repository:

https://github.com/Sumant3086/ProxyMukt-Attendance-System-

The repository is organised into two primary directories: /client (React frontend) and /server (Node.js/Express backend). The /server directory contains the following subdirectories: /routes (48 Express route handlers), /controllers (request processing logic), /services (six-layer pipeline implementation), /models (Mongoose schema definitions), and /middleware (JWT verification, rate limiting, Helmet security headers).

11.2 CRITICAL CODE EXCERPTS

11.2.1 HMAC-SHA256 QR Token Generation

```
// services/qrTokenService.js
const crypto = require('crypto');
const SECRET = process.env.HMAC_SECRET_KEY;
function generateQRToken(sessionId) {
  const timestamp = Date.now();
  const payload = `${sessionId}:${timestamp}`;
  const sig = crypto.createHmac('sha256',
SECRET).update(payload).digest('base64');
  return `${sig}:${timestamp}`;
}
```

11.2.2 Haversine Geofencing Validation

```
// services/geofenceService.js
function haversineDistance(lat1, lon1, lat2, lon2) {
  const R = 6371000; // Earth radius in metres
  const dLat = (lat2-lat1)*Math.PI/180;
  const dLon = (lon2-lon1)*Math.PI/180;
  const a = Math.sin(dLat/2)**2 + Math.cos(lat1*Math.PI/180) *
            Math.cos(lat2*Math.PI/180) * Math.sin(dLon/2)**2;
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
}
```

12. BIBLIOGRAPHY

The following references are cited in this report following the IEEE citation style.

[1] J. Gubbels, C. E. van der Put, and M. Assink, "Risk Factors for School Absenteeism and Dropout: A Meta-Analytic Review," Journal of Youth and Adolescence, vol. 48, no. 9, pp. 1637–1667, 2019, doi: 10.1007/s10964-019-01072-5.

[2] M. Kumar, A. J. Singh, and D. Handa, "Literature Survey on Student Performance Prediction in Education using Data Mining Techniques," International Journal of Education and Management Engineering, vol. 7, no. 6, pp. 40–49, 2017, doi: 10.5815/ijeme.2017.06.05.

[3] M. M. Aulia, N. Saha, and Md. M. Rahman, "Protected QR Code-based Anti-counterfeit System," Procedia CIRP, vol. 138, pp. 322–327, 2026, doi: 10.1016/j.procir.2026.01.056.

[4] M. P. Surve, M. K. Patwa, and M. M. Singh, "Geo-Location Attendance Tracking," Journal of Emerging Technologies and Innovative Research (JETIR), vol. 12, 2025. [Online]. Available: www.jetir.org

[5] I. Eweoya et al., "Design and Implementation of a University Attendance Management System Using Geo-Fencing," Asian Journal of Computer Science and Technology, vol. 14, no. 1, pp. 28–46, 2025, doi: 10.70112/AJCST-2025.14.1.4323.

[6] R. Baadkar, H. K. Soi, A. Kedare, and P. Bide, "Edu-Fy: Attendance Management and Attention Tracking System Using Deep Learning," in Proc. IEEE 4th International Conference for Advancement in Technology (ICONAT), 2025, pp. 1–6, doi: 10.1109/ICONAT66879.2025.11362872.

[7] S. Grey and N. Gordon, "Approaches to Measuring Attendance and Engagement," New Directions in the Teaching of Physical Sciences, vol. 13, no. 1, 2018, doi: 10.29311/ndtps.v0i13.2767.

[8] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[9] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," IEEE Internet Computing, vol. 14, no. 6, pp. 80–83, 2010, doi: 10.1109/MIC.2010.145.

[10] K. Chodorow, MongoDB: The Definitive Guide, 2nd ed. Sebastopol, CA: O'Reilly Media, 2013.

[11] IETF, "RFC 2104 — HMAC: Keyed-Hashing for Message Authentication," 1997. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc2104

[12] C. M. Gutierrez and J. M. Turner, "FIPS PUB 198-1: The Keyed-Hash Message Authentication Code (HMAC)," National Institute of Standards and Technology, 2008.

[13] F. Martin and D. U. Bolliger, "Engagement Matters: Student Perceptions on Engagement Strategies in Online Learning," Online Learning, vol. 22, no. 1, pp. 205–222, 2018, doi: 10.24059/olj.v22i1.1092.

[14] National Forum for Education Statistics (NFES), "Forum Guide to Attendance, Participation, and Engagement Data in Virtual and Hybrid Learning Models," 2021.

[15] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, 1999.

[16] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015, doi: 10.17487/RFC7519.

[17] P. Lubbers, B. Albers, and F. Salim, Pro HTML5 Programming. New York, NY: Apress, 2011, doi: 10.1007/978-1-4302-3865-2.

[18] A. K. Jain, R. P. W. Duin, and J. Mao, "Statistical Pattern Recognition: A Review," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 1, pp. 4–37, 2000, doi: 10.1109/34.824819.

[19] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN: Wiley, 2014.

[20] J. A. Whittaker, "What Is Software Testing? And Why Is It So Hard?" IEEE Software, vol. 17, no. 1, pp. 70–79, 2000, doi: 10.1109/52.819971.

[21] A. Dankar and P. P. Kundapur, "Automated Mobile Attendance System (AMAS)," in Proc. 6th IEEE International Conference on Advances in Computing, Communication and Control (ICAC3), 2019, doi: 10.1109/ICAC347590.2019.9036787.

[22] K. P. Borse et al., "Geo-Location Based Attendance Tracking Software Using Geofencing: A Grid-Based Approach," International Journal of Scientific

Development and Research (IJSDR), vol. 11, 2026. [Online]. Available: www.ijsdr.org

[23] P. Laperdrix, W. Rudametkin, and B. Baudry, "Beauty and the Beast: Diverting Modern Web Browsers to Build Unique Browser Fingerprints," in Proc. IEEE Symposium on Security and Privacy (SP), 2016, pp. 878–894, doi: 10.1109/SP.2016.57.

[24] D. R. Garrison and N. D. Vaughan, Blended Learning in Higher Education: Framework, Principles, and Guidelines. San Francisco, CA: Jossey-Bass, 2008.